SS*ZG681 / SE*ZG681: Advanced Cyber Security

Comprehensive Situated Learning Assignment (WILP) — Premium Student Handbook, Templates, Gold Solution, GitHub Starter, PlantUML, and Assessment Toolkit

Birla Institute of Technology & Science, Pilani
Work Integrated Learning Programmes Division
Second Semester 2024–2025

Document Purpose

This single unified Markdown document is the complete student-facing assignment handbook. It includes:

- Full assignment brief and submission expectations
- C4 Model architecture requirements
- Diagram-as-code requirements using PlantUML
- Detailed templates for every submission section
- Airline/Airbus-inspired sample projects for students who cannot disclose corporate systems
- A full gold-standard solved example
- MITRE-aligned attack graph templates
- Embedded CVSS calculator tables
- Embedded instructor grading rubric and grading sheet
- Viva/presentation outline
- Student GitHub starter repository template
- Automated grading helper scripts
- Academic integrity and originality-safe project variation guidance
- Optional enrichment activities for a memorable learning experience

> **Important confidentiality note:** Students must not disclose proprietary information, customer data, internal IP addresses, credentials, internal hostnames, unpublished designs, confidential processes, or any data restricted by their employer. When in doubt, abstract the system and use safe placeholders.

Table of Contents

1. Assignment Overview

This assignment requires each student to conduct an enterprise-grade cyber security architecture evaluation of a complex system. The system may be drawn from the student's workplace, provided that confidentiality rules are strictly followed. If the student's organization does not allow the use of corporate systems, the student may choose one of the airline/Airbus-inspired safe simulation projects provided in this document.

The expected output is a comprehensive engineering report that demonstrates practical understanding of:

- System architecture and security boundaries
- Data classification and regulatory exposure
- CIA triad analysis
- Multi-stage threat modeling
- Attack graph construction
- Defense-in-depth security design
- CVSS-based risk quantification
- Enterprise implementation constraints
- High-impact remediation planning

This is not a theoretical essay. It is a professional security engineering blueprint.

---

2. Assignment Information

Item	Details
Course Title and Code	Cyber Security — SS*ZG681 / SE*ZG681
Credit Structure	4 Credit Units — 3-1-0 Model
Evaluation Component	EC-1 — Expanded Situated Learning Project
Weightage	30 Marks
Submission Format	Comprehensive Engineering Report in PDF, supported by diagrams-as-code and GitHub repository evidence
Recommended Report Length	12–20 pages for standard submission; longer submissions are acceptable if well structured
Diagram Standard	C4 Model
Diagramming Approach	Diagram-as-code; PlantUML preferred
Repository Requirement	GitHub repository or equivalent Git-based version-controlled submission

---

3. Learning Objectives

By completing this assignment, students will be able to:

1. **Deconstruct and audit enterprise systems** using architecture reasoning, trust boundaries, data flows, and the CIA triad.
2. **Model realistic cyber threats** using multi-stage attacker progression and MITRE ATT&CK-style thinking.
3. **Design defense-in-depth controls** across cryptographic, network, application, endpoint, operational, and governance layers.
4. **Quantify risk using CVSS v3.1** and communicate technical risks to leadership.
5. **Balance ideal security design against real enterprise constraints** such as legacy systems, downtime windows, certification, cost, usability, and operational friction.
6. **Use modern engineering practices** such as GitHub, Markdown, PlantUML, and reproducible documentation.

---

4. Core Concept

The assignment centers on:

> Production-Grade Defense-in-Depth Implementation and Threat Modeling

Students must model and analyze a functional enterprise system, expose structural security weaknesses, and engineer a multi-layered security architecture covering:

- Data and cryptographic controls
- Network and perimeter controls
- Host, application, and endpoint controls
- Operational, managerial, and monitoring controls
- Compliance and governance controls

---

5. Scope and Constraints

The selected system must meet the following complexity parameters:

5.1 Multi-Component Architecture

The system should include multiple interacting components such as:

- Microservices
- APIs
- Message queues
- Databases
- Cloud infrastructure
- IoT/edge devices
- External vendor integrations
- Analytics pipelines
- Identity providers

5.2 Regulatory or Compliance Surface

The system should process or influence data governed by compliance requirements such as:

- GDPR
- Digital Personal Data Protection Act / DPDP
- PCI-DSS
- HIPAA
- SOC 2
- ISO 27001
- Aviation safety and operational compliance requirements
- Internal enterprise security policies

5.3 Operational Relevance

The system should be a realistic enterprise system. Trivial examples such as a single-page web app, hello-world API, or isolated lab script are not sufficient unless significantly expanded into a realistic architecture.

---

6. Confidentiality and Safe Abstraction Rules

Students working in sensitive organizations such as airlines, aerospace, aviation suppliers, defense contractors, financial institutions, or healthcare organizations must follow strict abstraction rules.

6.1 Do Not Disclose

Do not include:

- Real IP addresses
- Internal DNS names
- Customer data
- Passenger data
- Employee data
- Credentials or tokens
- Real vulnerability findings not approved for disclosure
- Proprietary algorithms
- Vendor-specific confidential integrations
- Internal incident details
- Security control gaps that could expose the employer

6.2 Safe Abstraction Examples

Sensitive Detail	Safe Replacement
`prod-airbus-eu-west-db-01.internal`	`Telemetry Database Cluster`
Real IP address	`Private subnet`
Real customer/passenger data	`Regulated user data`
Actual vendor name	`External maintenance partner`
Internal proprietary protocol	`Secure telemetry protocol`
Real exploit detail	`Input validation weakness`

6.3 Acceptable Alternative

If the employer does not permit using workplace systems, choose one of the provided simulation projects and clearly state:

> “This submission uses a realistic simulated enterprise architecture inspired by the airline/aerospace industry. No proprietary organizational information is used.”

7. Submission Format and LMS/GitHub Workflow

7.1 Required Deliverables

Students must submit:

1. Final report as PDF
2. GitHub repository link or archive
3. PlantUML source files for diagrams
4. CVSS scoring table included in the report
5. Attack graph descriptions and/or diagram source
6. Optional: presentation slides for viva

7.2 Recommended GitHub Repository Structure

```
advanced-cyber-security-assignment/
├── README.md
├── report/
│   ├── final-report.md
│   └── final-report.pdf
├── diagrams/
│   ├── c4-context.puml
│   ├── c4-container.puml
│   ├── c4-component.puml
│   ├── attack-graph-1.puml
│   └── attack-graph-2.puml
├── risk/
│   ├── cvss-analysis.md
│   └── risk-register.csv
├── evidence/
│   ├── screenshots-placeholder.md
│   └── assumptions.md
├── scripts/
│   ├── grade_report.py
│   └── check_structure.py
└── presentation/
    └── viva-outline.md
```

7.3 GitHub Expectations

Students should show:

- Meaningful commits
- Clear directory structure
- Version history
- Diagram source files
- Markdown report source
- Traceability between report and diagrams

8. Architecture Requirements: C4 Model

All architecture diagrams must follow the C4 Model. Students must include at least:

1. **C4 Level 1: System Context Diagram**
2. **C4 Level 2: Container Diagram**
3. **C4 Level 3: Component Diagram**

C4 Level 4 code/class-level diagram is optional and should be included only if useful.

8.1 C4 Level 1 — System Context

Purpose: Show the system as a black box and identify external actors/systems.

Must include:

- Primary users
- External enterprise systems
- External data providers
- Regulatory or governance systems if relevant
- Major trust boundaries

8.2 C4 Level 2 — Container

Purpose: Show major runtime containers.

Examples:

- Web app
- API gateway
- Microservices
- Database
- Message queue
- Edge gateway
- Data lake
- SIEM
- Identity provider

8.3 C4 Level 3 — Component

Purpose: Show internal components of important containers.

Examples:

- Authentication component
- Telemetry validation component
- Data ingestion worker
- Alerting engine
- Authorization policy module
- Audit logger

8.4 C4 Level 4 — Code or Class View Optional

Use this only if a small security-sensitive part of the design deserves deeper explanation, such as:

- Token validation logic
- Secure input validation
- Cryptographic key rotation flow
- Policy enforcement module

---

9. Diagram-as-Code Requirement: PlantUML Preferred

Students must use diagram-as-code. PlantUML is preferred because it is version-controllable, reviewable, and suitable for GitHub workflows.

Acceptable options:

1. PlantUML — preferred
2. Mermaid — acceptable
3. Structurizr DSL — acceptable
4. Draw.io — allowed only if source file is versioned and exported; not preferred

9.1 Required Diagram Files

Minimum required files:

```
 diagrams/c4-context.puml
diagrams/c4-container.puml
diagrams/c4-component.puml
diagrams/attack-graph-1.puml
diagrams/attack-graph-2.puml
```

9.2 Diagram Quality Checklist

Each diagram should include:

- Title
- Actors
- Containers/components
- Trust boundaries
- Data flows
- Protocols where relevant
- Security controls where relevant

- Legend or notes for assumptions

---

10. Required Repository Structure

Students may use the following starter structure.

```
student-project/
├── README.md
├── report/
│   ├── final-report.md
│   ├── references.md
│   └── final-report.pdf
├── diagrams/
│   ├── c4-context.puml
│   ├── c4-container.puml
│   ├── c4-component.puml
│   ├── c4-code-optional.puml
│   ├── attack-graph-scenario-1.puml
│   ├── attack-graph-scenario-2.puml
│   └── did-architecture.puml
├── risk/
│   ├── cvss-table.md
│   ├── risk-register.csv
│   └── gap-analysis.md
├── templates/
│   ├── data-classification-matrix.md
│   ├── cia-analysis.md
│   └── defense-in-depth-template.md
├── scripts/
│   ├── check_structure.py
│   ├── grade_report.py
│   └── README.md
└── presentation/
    └── viva-presentation-outline.md
```

---

11. Section B Student Submission Structure

B.1 Submission Form

Students must fill:

- Student full name
- Student ID
- Years of professional experience
- Current role
- Industry domain
- Selected system name
- Whether the system is real, anonymized, or simulated

B.2 Anchor — System and Threat Surface Blueprinting — 5 Marks

Students must provide:

1. Architecture narrative
2. C4 diagrams
3. Data flow mapping
4. Trust boundaries
5. Data classification matrix
6. Compliance/regulatory mapping

B.3 Apply — Advanced Cyber Security Engineering Pipeline — 15 Marks

Task 1: CIA Asset Valuation and Cryptographic Policy — 3 Marks

Students must provide:

- Confidentiality failure scenario
- Integrity failure scenario
- Availability failure scenario
- Business impact
- Security model selection
- Cryptographic design

Task 2: Multi-Stage Threat Vectors and Attack Graphs — 5 Marks

Students must provide:

- At least two complex attack scenarios
- Attack progression graph
- Threat actor goal
- Affected components
- Entry points
- Lateral movement assumptions
- Detection opportunities

Task 3: Defense-in-Depth Infrastructure Design — 7 Marks

Students must design controls across:

1. Cryptographic and data controls
2. Network and perimeter controls
3. Host, application, and endpoint controls
4. Operational, managerial, and monitoring controls

B.4 Analyse — Gap Analysis and CVSS Risk Quantification — 5 Marks

Students must provide:

- Current vs proposed architecture comparison
- Organizational trade-off analysis
- CVSS v3.1 vector strings for top two vulnerabilities
- Risk register
- Leadership-level interpretation

B.5 Reflect — Enterprise Constraints and Lifecycle Maintenance — 5 Marks

Students must provide:

- Technical constraints
- Organizational constraints
- Cost/TCO implications
- Developer experience impact
- Deployment sequencing
- Single highest-impact immediate remediation

---

12. Detailed Student Templates

12.1 Cover Page Template

```
 # Advanced Cyber Security Assignment
## SS*ZG681 / SE*ZG681

**Student Name:**
**Student ID:**
**Professional Experience:**
**Current Role:**
**Industry Domain:**
**System Selected:**
**System Type:** Real / Anonymized / Simulated
**Confidentiality Statement:** This report does not disclose proprietary or sensitive organizational information.
```

12.2 Executive Summary Template

```
 # Executive Summary

This report evaluates the security posture of [system name], a [brief system type] used for [business purpose]. The system includes [maj
```

12.3 B2 Architecture Template

```
 # B.2 Anchor — System and Threat Surface Blueprinting

## 1. System Description
[Describe the system purpose, users, business function, and operational importance.]

## 2. Runtime Architecture
[Describe major components, deployment model, cloud/on-prem/edge components, and integrations.]

## 3. C4 Diagrams
- Context diagram: `diagrams/c4-context.puml`
- Container diagram: `diagrams/c4-container.puml`
- Component diagram: `diagrams/c4-component.puml`

## 4. Data Flow Mapping
[Provide table of sources, destinations, protocols, data types, trust boundaries.]

## 5. Data Classification Matrix
[Insert matrix.]

## 6. Regulatory and Compliance Boundary
[Map data and processes to relevant regulations/policies.]
```

12.4 B3 CIA and Cryptographic Policy Template

```
 # B.3 Task 1 — CIA Asset Valuation and Cryptographic Policy

## Confidentiality Failure Scenario
[Explain what happens if sensitive data is exposed.]

## Integrity Failure Scenario
[Explain what happens if data is modified, spoofed, poisoned, or corrupted.]

## Availability Failure Scenario
[Explain what happens if system becomes unavailable.]

## Business Impact
[Financial, safety, operational, legal, reputational impact.]

## Security Models
- RBAC / ABAC / Bell-LaPadula / Biba / Zero Trust
[Justify selection.]

## Cryptographic Policy
- Data at rest:
- Data in transit:
- Data in use:
- Key management:
- Certificate lifecycle:
- Logging and audit:
```

12.5 B3 Threat Modeling Template

```
# B.3 Task 2 — Threat Modeling and Attack Graphs

## Threat Scenario 1: [Name]

### Threat Actor
[Insider / external attacker / partner compromise / supply-chain actor]

### Objective
[Exfiltration / disruption / manipulation / fraud / persistence]

### Attack Graph
Initial Access → Execution → Persistence → Privilege Escalation → Lateral Movement → Action on Objectives

### Affected Assets
[List assets.]

### Detection Opportunities
[List logs, alerts, telemetry.]

### Existing Weaknesses
[List weaknesses.]

## Threat Scenario 2: [Name]
[Repeat same structure.]
```

12.6 B3 Defense-in-Depth Template

```
# B.3 Task 3 — Defense-in-Depth Architecture

## Layer 1: Cryptographic and Data Security Controls
[Encryption, signing, KMS, tokenization, hashing, data minimization.]

## Layer 2: Network and Perimeter Controls
[Segmentation, ZTNA, WAF, mTLS, firewalls, private endpoints.]

## Layer 3: Host, Application, and Endpoint Controls
[Secure coding, scanning, runtime protection, EDR, container controls.]

## Layer 4: Operational, Managerial, and Monitoring Controls
[SIEM, SOAR, incident response, audit, governance, runbooks.]

## Control-to-Threat Mapping
[Map each control to threats from Task 2.]
```

12.7 B4 Gap Analysis Template

```
# B.4 Analyse — Gap Analysis and CVSS Risk Quantification

## Current State
[Describe current production or simulated baseline.]

## Proposed State
[Describe target defense architecture.]

## Gap Table
[Insert table.]

## CVSS Scoring for Top Two Vulnerabilities
[Insert vectors and explanation.]

## Leadership Interpretation
[Explain risks in language suitable for decision makers.]
```

12.8 B5 Reflection Template

```
# B.5 Reflect — Enterprise Constraints and Lifecycle Maintenance

## Technical Friction
[Legacy systems, downtime, certification, data migration, tool integration.]

## Organizational Friction
[Budget, ownership, team skills, policy resistance, vendor dependency.]

## TCO Considerations
[Licensing, cloud costs, compute overhead, operations headcount.]

## Developer Experience Impact
[CI/CD changes, approval workflows, scanning overhead.]

## Immediate High-Impact Remediation
[Select one change. Explain risk reduction vs implementation effort.]
```

13. Data Classification Matrix Template

Asset / Data Element	Description	Criticality	Confidentiality Impact	Integrity Impact	Availability Impact	Owner	Regulation / Policy	Retention	Controls Required
Example: Aircraft telemetry	Engine and sensor data	High	Medium	High	High	Engineering Ops	Aviation safety policy	7 years	TLS, signing, validation, immutable logs
Example: Passenger PII	Name, contact, booking info	High	High	Medium	Medium	Data Protection Officer	GDPR/DPDP	As per policy	Encryption, masking, access control
Example: Maintenance work order	Repair and inspection records	High	Medium	High	Medium	Maintenance Lead	Internal safety policy	10 years	Integrity checks, audit trail

14. MITRE-Aligned Threat Modeling and Attack Graph Templates

> This assignment uses MITRE-style attacker progression as a learning structure. Students do not need to map every step to exact MITRE technique IDs, but should show realistic attack progression.

14.1 Generic Attack Graph Template

```
Reconnaissance
→ Initial Access
→ Execution
→ Persistence
→ Privilege Escalation
→ Defense Evasion
→ Credential Access
→ Discovery
→ Lateral Movement
→ Collection
→ Command and Control
→ Exfiltration / Impact
```

14.2 Attack Graph Table Template

Stage	Attacker Action	Target Component	Weakness Exploited	Evidence/Logs	Detection Control	Mitigation
Reconnaissance	Identify public API	API Gateway	Public endpoint metadata	Access logs	WAF analytics	Reduce exposed metadata
Initial Access	Abuse weak auth	Login/API	Weak token validation	Auth logs	SIEM alert	Strong auth, mTLS
Execution	Inject malicious payload	Ingestion service	Input validation gap	App logs	Runtime detection	Schema validation
Lateral Movement	Access data lake	Cloud IAM	Excessive privileges	IAM logs	Cloud posture alerts	Least privilege
Impact	Poison analytics	ML/analytics pipeline	No integrity checks	Pipeline logs	Data quality alarms	Signed telemetry

14.3 PlantUML Attack Graph Template

```
@startuml
skinparam shadowing false
skinparam backgroundColor #FFFFFF
skinparam activity {
  BackgroundColor #F8F9FA
  BorderColor #333333
}

title MITRE-Aligned Attack Graph Template

start
:Reconnaissance;
:Initial Access;
:Execution;
:Persistence;
:Privilege Escalation;
:Discovery;
:Lateral Movement;
:Collection;
:Exfiltration / Impact;
stop
@enduml
```

15. Defense-in-Depth Design Template

15.1 Control Layers

Layer	Control Family	Example Controls	Evidence Expected
1	Data and Cryptography	TLS 1.3, AES-256-GCM, KMS, signing, tokenization	Crypto policy, key lifecycle, data flow
2	Network and Perimeter	Segmentation, mTLS, WAF, ZTNA, private endpoints	Network diagram, trust boundaries
3	Host/Application/Endpoint	Secure coding, SAST/DAST, container scanning, EDR	CI/CD controls, scan policy
4	Operations/Governance	SIEM, SOAR, runbooks, audit, incident response	Alert rules, escalation path

15.2 Control-to-Threat Traceability Matrix

Threat Scenario	Attack Stage	Proposed Control	Layer	Prevent / Detect / Respond	Residual Risk
API exploit	Initial access	WAF + API schema validation	Network/Application	Prevent	Medium
API exploit	Lateral movement	IAM least privilege	Operations/Application	Prevent	Low
Sensor spoofing	Execution	Signed telemetry	Data	Prevent/Detect	Low
Ransomware	Impact	Immutable backups	Operations	Respond/Recover	Medium

16. CVSS v3.1 Risk Quantification Framework

16.1 CVSS Base Metric Table

Metric	Abbreviation	Values	Student Selection	Explanation
Attack Vector	AV	N, A, L, P		Network, Adjacent, Local, Physical
Attack Complexity	AC	L, H		Low or High complexity
Privileges Required	PR	N, L, H		None, Low, High
User Interaction	UI	N, R		None or Required
Scope	S	U, C		Unchanged or Changed
Confidentiality	C	H, L, N		High, Low, None
Integrity	I	H, L, N		High, Low, None
Availability	A	H, L, N		High, Low, None

16.2 Temporal Metric Table

Metric	Abbreviation	Values	Student Selection	Explanation
Exploit Code Maturity	E	X, H, F, P, U		Maturity of exploit
Remediation Level	RL	X, U, W, T, O		Availability of fix
Report Confidence	RC	X, C, R, U		Confidence in report

16.3 Environmental Metric Table

Metric	Abbreviation	Values	Student Selection	Explanation
Confidentiality Requirement	CR	X, H, M, L		Business importance
Integrity Requirement	IR	X, H, M, L		Business importance
Availability Requirement	AR	X, H, M, L		Business importance

Metric	Abbreviation	Values	Student Selection	Explanation

16.4 CVSS Worked Example

Vulnerability	Vector	Base Score	Severity	Explanation
Public API injection	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H	9.8	Critical	Network exploitable, no privileges, high impact
Sensor spoofing over adjacent link	AV:A/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:H	8.1	High	Adjacent access, high integrity and availability impact

> Students may use the official CVSS calculator or calculate manually, but the vector string and reasoning must be included in the report.

---

17. Evaluation Rubric — 30 Marks

17.1 Summary Rubric

Component	Criteria	Marks
B.2 Anchor	System and threat surface blueprinting	5
B.3 Apply	CIA, threat modeling, attack graphs, defense-in-depth	15
B.4 Analyse	Gap analysis and CVSS quantification	5
B.5 Reflect	Constraints, trade-offs, lifecycle, remediation	5
Total	Comprehensive engineering and security evaluation	30

17.2 Detailed Rubric

B.2 Anchor — 5 Marks

Performance Level	Description	Marks Range
Excellent	Complete C4 diagrams, precise data flows, trust boundaries, classification, compliance mapping	4.5–5
Good	Clear architecture and data flows but minor gaps in classification or compliance	3.5–4.4
Satisfactory	Basic architecture present, limited security boundary detail	2.5–3.4
Weak	Vague architecture, missing diagrams or data classification	0–2.4

B.3 Apply — 15 Marks

Subcomponent	Criteria	Marks
CIA and crypto policy	Concrete CIA scenarios, business impact, security models, crypto policy	3
Threat modeling	Two realistic multi-stage attack graphs with technical depth	5
Defense-in-depth	Four-layer control architecture with traceability to threats	7

B.4 Analyse — 5 Marks

Performance Level	Description	Marks Range

Performance Level	Description	Marks Range
Excellent	Strong current-vs-target analysis, CVSS vectors, leadership interpretation	4.5–5
Good	Good gap analysis and CVSS but limited business interpretation	3.5–4.4
Satisfactory	Basic gap analysis, incomplete scoring explanation	2.5–3.4
Weak	No quantitative risk analysis	0–2.4

B.5 Reflect — 5 Marks

Performance Level	Description	Marks Range
Excellent	Realistic enterprise constraints, TCO, deployment plan, clear high-impact remediation	4.5–5
Good	Good reflection but limited prioritization detail	3.5–4.4
Satisfactory	General constraints listed but weak remediation reasoning	2.5–3.4
Weak	Superficial reflection	0–2.4

---

18. Instructor Auto-Grading Sheet Template

This table can be copied into Excel.

Student ID	Student Name	B2 Architecture 5	B3.1 CIA 3	B3.2 Threats 5	B3.3 Defense 7	B4 Analysis 5	B5 Reflection 5	Total 30	Comments
								`=SUM(C2:H2)`	

Suggested Excel formulas:

```
=SUM(C2:H2)
```

```
=IF(I2>=27,"Excellent",IF(I2>=22,"Good",IF(I2>=16,"Satisfactory","Needs Improvement")))
```

---

19. Viva/Presentation Template

Students may use 7–10 slides.

Slide 1 — Title

- Course and assignment title
- Student name and ID
- System selected
- Real/anonymized/simulated declaration

Slide 2 — System Context

- Business purpose
- Users
- External systems
- Key risks

Slide 3 — C4 Architecture

- Context diagram
- Container diagram
- Key trust boundaries

Slide 4 — Data Classification

- Critical data assets
- Compliance implications
- CIA impact summary

Slide 5 — Threat Scenario 1

- Attack graph
- Key vulnerability
- Impact

Slide 6 — Threat Scenario 2

- Attack graph
- Key vulnerability
- Impact

Slide 7 — Defense-in-Depth Design

- Controls across four layers
- Control-to-threat mapping

Slide 8 — CVSS and Gap Analysis

- Top two risks
- Scores
- Current vs proposed state

Slide 9 — Remediation Roadmap

- Immediate fix
- Medium-term improvements
- Long-term governance

Slide 10 — Reflection and Lessons Learned

- Constraints
- Trade-offs
- What changed in your thinking

---

20. Airline/Airbus-Inspired Sample Project Bank

These examples are safe simulations and do not represent actual Airbus systems.

20.1 Mechanical Engineering Track — Aircraft Predictive Maintenance Platform

Scenario

A fleet maintenance platform collects vibration, temperature, pressure, and engine health data from aircraft components. Data is processed at the edge and sent to a cloud analytics system for predictive maintenance.

Architecture

```
Sensors → Edge Gateway → Message Broker → Cloud Data Lake → ML Model → Maintenance Dashboard
```

Suitable Threats

- Sensor data spoofing
- Data poisoning attack against predictive model
- Ransomware affecting maintenance scheduling
- Compromised edge gateway

Suggested Security Focus

- Telemetry integrity
- Signed sensor data
- Edge device hardening
- Immutable logs
- ML data validation

20.2 Aeronautical Engineering Track — Flight Operations Communication Platform

Scenario

A flight operations platform exchanges flight plan updates, weather updates, and operational messages between aircraft systems, ground control interfaces, and airline operations.

Architecture

```
Aircraft System → Secure Communication Gateway → Ground Station → Operations Platform → Dispatch Dashboard
```

Suitable Threats

- Signal spoofing
- Message replay
- Unauthorized operational update
- Availability attack on ground communication

Suggested Security Focus

- Message authentication
- Replay prevention
- Availability and redundancy
- Secure gateway design

20.3 Software Engineering Track — Airline Passenger Management Platform

Scenario

A cloud-native passenger platform manages booking, check-in, boarding pass issuance, loyalty profile lookup, and partner integrations.

Architecture

```
Mobile/Web App → API Gateway → Identity Service → Booking Service → Payment Service → Passenger Database
```

Suitable Threats

- API session hijacking
- SQL injection chaining
- OAuth misconfiguration
- Payment data exposure
- Partner API compromise

Suggested Security Focus

- API security
- OAuth/OIDC hardening
- WAF
- Least privilege
- Token lifecycle
- PCI-DSS considerations

20.4 Electronics/Embedded Systems Track — Avionics Health Monitoring Gateway

Scenario

An embedded gateway aggregates health status from avionics subsystems and forwards summarized operational status to a maintenance monitoring backend.

Suitable Threats

- Firmware tampering
- Malicious update package
- Physical interface misuse
- Weak device identity

Suggested Security Focus

- Secure boot
- Firmware signing
- Device identity
- Hardware root of trust

20.5 Data/AI Track — Flight Delay Prediction and Operations Optimization Platform

Scenario

A data platform uses weather, airport congestion, fleet status, crew availability, and maintenance signals to predict flight delays and optimize operational decisions.

Suitable Threats

- Data poisoning
- Model drift manipulation
- Unauthorized data lake access
- Insider manipulation of training data

Suggested Security Focus

- Data lineage
- ML pipeline integrity
- Access control
- Model monitoring

---

21. Full Gold-Standard Solved Example

21.1 Title

Security Architecture Evaluation of an Aircraft Telemetry Data Platform

21.2 Confidentiality Statement

This is a simulated Airbus-inspired aviation telemetry platform. It does not represent an actual Airbus system. All component names, flows, risks, and controls are generalized for academic learning.

21.3 Executive Summary

The Aircraft Telemetry Data Platform is a distributed aviation analytics system that collects aircraft sensor data, processes it at an edge gateway, transmits it through secure communication channels, stores it in a cloud data lake, and supports predictive maintenance analytics. The system is operationally critical because telemetry integrity directly influences maintenance decisions, aircraft availability, safety monitoring, and fleet reliability.

This report evaluates the platform using the CIA triad, C4 architecture modeling, data classification, MITRE-style threat modeling, CVSS risk scoring, and defense-in-depth engineering. Two major attack scenarios are analyzed: sensor data spoofing leading to predictive maintenance data poisoning, and API compromise leading to unauthorized telemetry exfiltration. A layered defense architecture is proposed, including signed telemetry, mTLS, zero-trust access, API gateway hardening, SIEM monitoring, immutable audit trails, and incident response automation.

The highest-priority remediation is to implement cryptographic telemetry signing at the edge gateway combined with device identity validation. This reduces the risk of false telemetry injection, improves forensic traceability, and strengthens trust in downstream analytics.

21.4 System Overview

The Aircraft Telemetry Data Platform collects telemetry from aircraft subsystems such as engines, landing gear, environmental systems, vibration sensors, temperature sensors, and avionics health indicators. Data is aggregated by an onboard or near-edge gateway, preprocessed, encrypted, and transmitted to a ground/cloud ingestion platform. The cloud platform stores raw and processed telemetry in a data lake and runs analytics pipelines to identify anomalies, predict maintenance needs, and generate operational dashboards.

21.5 Business and Operational Importance

Business Function	Importance
Predictive maintenance	Reduces unscheduled downtime and improves fleet availability
Safety monitoring	Helps detect abnormal component behavior
Operational analytics	Supports reliability engineering and operational planning
Regulatory evidence	Maintains audit trails for maintenance and operational decisions
Cost optimization	Avoids unnecessary part replacement and reduces delays

21.6 C4 Level 1 — System Context Diagram

```
@startuml
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Context.puml

LAYOUT_WITH_LEGEND()

title C4 Level 1 - Aircraft Telemetry Data Platform Context

Person(maintenanceEngineer, "Maintenance Engineer", "Reviews aircraft health and maintenance recommendations")
Person(operationsManager, "Operations Manager", "Monitors fleet operational readiness")
Person(securityAnalyst, "Security Analyst", "Monitors security events and incidents")

System(atdp, "Aircraft Telemetry Data Platform", "Collects, processes, stores, and analyzes aircraft telemetry")
System_Ext(aircraftSystems, "Aircraft Sensor and Subsystem Network", "Generates telemetry and health data")
System_Ext(identityProvider, "Enterprise Identity Provider", "Provides authentication and authorization")
System_Ext(siem, "Enterprise SIEM/SOAR", "Security monitoring and response")
System_Ext(partnerSystem, "Approved Maintenance Partner", "Receives selected maintenance work orders")

Rel(aircraftSystems, atdp, "Sends telemetry", "Secure telemetry protocol / TLS")
Rel(maintenanceEngineer, atdp, "Views recommendations", "HTTPS")
Rel(operationsManager, atdp, "Views fleet dashboards", "HTTPS")
Rel(securityAnalyst, siem, "Investigates alerts", "SIEM console")
Rel(atdp, identityProvider, "Authenticates users", "OIDC/SAML")
Rel(atdp, siem, "Sends audit and security logs", "Syslog/API")
Rel(atdp, partnerSystem, "Shares approved maintenance tasks", "Partner API")

@enduml
```

21.7 C4 Level 2 — Container Diagram

```
@startuml
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Container.puml

LAYOUT_WITH_LEGEND()

title C4 Level 2 - Aircraft Telemetry Data Platform Containers

Person(maintenanceEngineer, "Maintenance Engineer")
Person(operationsManager, "Operations Manager")

System_Boundary(atdp, "Aircraft Telemetry Data Platform") {
    Container(edgeGateway, "Edge Telemetry Gateway", "Embedded Linux / hardened gateway", "Aggregates and signs telemetry before transmi
    Container(apiGateway, "API Gateway", "Managed API Gateway", "Exposes secure APIs and enforces auth, rate limits, and WAF policies")
    Container(ingestionService, "Telemetry Ingestion Service", "Microservice", "Validates schema, verifies signatures, and writes events
    Container(messageBroker, "Telemetry Message Broker", "Kafka / Event Streaming", "Buffers telemetry events")
    Container(dataLake, "Telemetry Data Lake", "Object Storage", "Stores raw and curated telemetry")
    Container(analyticsEngine, "Predictive Analytics Engine", "Spark / ML Pipeline", "Detects anomalies and predicts maintenance needs")
    Container(dashboard, "Maintenance Dashboard", "Web Application", "Displays fleet health and recommendations")
    Container(auditService, "Audit and Security Logging Service", "Logging Pipeline", "Collects audit, access, and security events")
}

System_Ext(idp, "Enterprise Identity Provider", "OIDC/SAML")
System_Ext(siem, "SIEM/SOAR", "Security monitoring")
System_Ext(partner, "Maintenance Partner API", "Approved external partner")

Rel(edgeGateway, ingestionService, "Sends signed telemetry", "mTLS + JSON/Protobuf")
Rel(ingestionService, messageBroker, "Publishes validated telemetry", "Kafka protocol")
Rel(messageBroker, dataLake, "Stores raw telemetry", "Connector")
Rel(dataLake, analyticsEngine, "Feeds analytics", "Batch/stream processing")
Rel(analyticsEngine, dashboard, "Provides recommendations", "Internal API")
Rel(maintenanceEngineer, dashboard, "Uses", "HTTPS")
Rel(operationsManager, dashboard, "Uses", "HTTPS")
Rel(dashboard, apiGateway, "Calls APIs", "HTTPS")
Rel(apiGateway, idp, "Validates identity", "OIDC")
Rel(auditService, siem, "Forwards logs", "API/Syslog")
Rel(apiGateway, partner, "Sends approved work orders", "mTLS API")

@enduml
```

21.8 C4 Level 3 — Component Diagram for Ingestion Service

```
@startuml
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Component.puml

LAYOUT_WITH_LEGEND()

title C4 Level 3 - Telemetry Ingestion Service Components

Container_Boundary(ingestion, "Telemetry Ingestion Service") {
    Component(receiver, "Telemetry Receiver", "REST/gRPC endpoint", "Receives telemetry from edge gateways")
    Component(schemaValidator, "Schema Validator", "Validation module", "Validates message structure and allowed ranges")
    Component(signatureVerifier, "Signature Verifier", "Crypto module", "Verifies telemetry signature and device identity")
    Component(replayDetector, "Replay Detector", "Stateful validation", "Rejects duplicate or stale telemetry messages")
    Component(policyEngine, "Policy Engine", "Authorization module", "Checks gateway permissions and data policy")
    Component(publisher, "Event Publisher", "Kafka producer", "Publishes validated telemetry events")
    Component(auditLogger, "Audit Logger", "Logging component", "Emits security and audit events")
}

Container(edgeGateway, "Edge Gateway", "Hardened device")
Container(messageBroker, "Message Broker", "Kafka")
Container(auditService, "Audit Service", "Logging pipeline")

Rel(edgeGateway, receiver, "Sends telemetry", "mTLS")
Rel(receiver, schemaValidator, "Passes message")
Rel(schemaValidator, signatureVerifier, "Validated structure")
Rel(signatureVerifier, replayDetector, "Verified signature")
Rel(replayDetector, policyEngine, "Fresh message")
Rel(policyEngine, publisher, "Authorized event")
Rel(publisher, messageBroker, "Publishes event")
Rel(auditLogger, auditService, "Sends audit logs")
Rel(receiver, auditLogger, "Logs request metadata")
Rel(signatureVerifier, auditLogger, "Logs verification failures")

@enduml
```

21.9 Data Flow Mapping

Source	Destination	Protocol	Data Type	Trust Boundary	Security Requirement
Aircraft sensors	Edge gateway	Field bus / internal subsystem protocol	Raw sensor telemetry	Aircraft subsystem boundary	Input sanity checks, local validation
Edge gateway	Ingestion service	mTLS over secure network	Signed telemetry	Aircraft-to-ground/cloud boundary	Mutual authentication, signing, replay protection
Ingestion service	Message broker	Kafka/internal secure transport	Validated telemetry event	Application boundary	Service identity, encryption in transit
Message broker	Data lake	Connector/API	Raw/curated telemetry	Data platform boundary	Encryption at rest, IAM least privilege
Data lake	Analytics engine	Batch/stream processing	Historical telemetry	Analytics boundary	Data lineage, integrity validation
Analytics engine	Dashboard	Internal API	Maintenance recommendations	App boundary	Authorization, audit logging
Dashboard	Users	HTTPS	Visualized results	User access boundary	SSO, MFA, RBAC
Platform services	SIEM	API/Syslog	Audit/security logs	Security operations boundary	Tamper-resistant logging

21.10 Data Classification Matrix

Asset / Data Element	Description	Criticality	Confidentiality	Integrity	Availability	Owner	Compliance / Policy	Controls Required

Asset / Data Element	Description	Criticality	Confidentiality	Integrity	Availability	Owner	Compliance / Policy	Controls Required
Engine telemetry	Vibration, temperature, pressure	High	Medium	High	High	Engineering Ops	Aviation safety policy	Signing, validation, encryption
Avionics health indicators	Health status and alerts	High	Medium	High	High	Flight Ops Engineering	Internal safety policy	Integrity checks, restricted access
Maintenance recommendation	Predicted failure or work order	High	Medium	High	Medium	Maintenance Lead	Maintenance governance	Audit trail, approval workflow
User identity data	Engineer login and role	Medium	High	Medium	Medium	IAM Team	Privacy/security policy	MFA, RBAC, logging
Partner work order data	Shared maintenance tasks	Medium	Medium	High	Medium	Partner Mgmt	Contractual controls	mTLS, data minimization
Security logs	Access and event logs	High	High	High	Medium	SOC	Security policy	Immutable logging, restricted access

21.11 CIA Analysis

CIA Dimension	Failure Scenario	Business / Safety Impact	Technical Consequence	Priority
Confidentiality	Telemetry and maintenance data exposed to unauthorized party	Competitive, safety, and operational exposure	Unauthorized knowledge of fleet health	Medium-High
Integrity	Attacker injects false telemetry indicating normal engine state	Incorrect maintenance decisions; safety risk	Analytics model produces false negatives	Critical
Availability	Telemetry ingestion pipeline unavailable during operations	Delayed maintenance decisions and fleet disruptions	Data loss or backlog	High

21.12 Security Models

Security Model	Use in Platform	Justification
RBAC	Dashboard and operational roles	Engineers, analysts, and partners need different permissions
ABAC	Fine-grained access based on aircraft fleet, region, role, data sensitivity	Better for complex enterprise environments
Biba Integrity Model	Telemetry ingestion and analytics	Integrity is critical for maintenance decisions
Zero Trust	Gateway, service, and user access	No implicit trust based on network location

21.13 Cryptographic Policy

Area	Policy
Data in transit	TLS 1.3 or mTLS for gateway-to-cloud and service-to-service communication
Data at rest	AES-256-GCM or cloud provider equivalent encryption for object storage, databases, and logs
Data in use	Restrict memory exposure; consider confidential computing for high-sensitivity analytics workloads
Telemetry authenticity	Edge gateway signs telemetry batches using device-bound private keys
Key management	Centralized KMS/HSM-backed key lifecycle, rotation, revocation

Area	Policy
Certificate management	Short-lived certificates, automated renewal, certificate pinning where appropriate
Auditability	Log all key usage, failed verification, and certificate anomalies

21.14 Threat Scenario 1 — Sensor Spoofing and Data Poisoning

Threat Summary

An attacker compromises or emulates a telemetry source and sends manipulated telemetry values to the edge gateway or ingestion service. If not detected, poisoned data enters the data lake and influences predictive maintenance models.

Attack Graph Table

Stage	Attacker Action	Target	Weakness	Evidence	Detection	Mitigation
Reconnaissance	Study telemetry format	Edge/ingestion interface	Predictable schema	Access attempts	API anomaly detection	Minimize exposed metadata
Initial Access	Connect rogue telemetry device	Edge network	Weak device identity	Unknown device logs	Device inventory alerts	Device certificates
Execution	Send plausible false readings	Edge gateway	Insufficient validation	Outlier readings	Data quality checks	Schema/range validation
Persistence	Continue low-volume poisoning	Telemetry stream	No behavioral baseline	Gradual drift	ML drift detection	Baseline anomaly detection
Lateral Movement	Influence analytics pipeline	Data lake/ML	Trust in upstream data	Model input records	Data lineage monitoring	Signed data lineage
Impact	Incorrect maintenance recommendation	Dashboard	Model poisoning	Recommendation change	Human review workflow	Approval and confidence scoring

PlantUML Attack Graph

```
@startuml
skinparam shadowing false
title Attack Scenario 1 - Sensor Spoofing and Data Poisoning

start
:Reconnaissance of telemetry format;
:Initial access through rogue or compromised telemetry source;
:Inject plausible false sensor readings;
:Maintain low-volume poisoning to avoid detection;
:Poison data lake and analytics pipeline;
:Generate incorrect maintenance recommendation;
stop
@enduml
```

21.15 Threat Scenario 2 — API Compromise and Telemetry Exfiltration

Threat Summary

An attacker exploits weak API validation or broken authorization to access telemetry data or maintenance records through the dashboard/API layer.

Attack Graph Table

Stage	Attacker Action	Target	Weakness	Evidence	Detection	Mitigation
Reconnaissance	Enumerate API endpoints	API gateway	Verbose errors	404/403 spikes	WAF analytics	Error normalization
Initial Access	Abuse weak token/session	API	Misconfigured auth	Failed auth logs	Auth anomaly alert	Strong OIDC validation
Execution	Submit malicious query/payload	API service	Input validation gap	App errors	DAST/SIEM alert	Schema validation
Privilege Escalation	Access broader telemetry data	IAM/API	Broken object-level auth	Unusual object access	IAM anomaly detection	ABAC and object checks
Collection	Export telemetry records	Data API	Excessive query access	Large downloads	DLP alert	Rate limits and query limits
Exfiltration	Transfer data externally	Network	Weak egress control	Egress logs	Egress anomaly detection	Egress filtering

PlantUML Attack Graph

```
@startuml
skinparam shadowing false
title Attack Scenario 2 - API Compromise and Telemetry Exfiltration

start
:Enumerate public API endpoints;
:Exploit weak authentication or token validation;
:Submit malicious API requests;
:Bypass object-level authorization;
:Collect telemetry and maintenance records;
:Exfiltrate data through API responses;
stop
@enduml
```

21.16 Defense-in-Depth Architecture

Layer 1 — Cryptographic and Data Security Controls

Control	Purpose	Threat Addressed
Signed telemetry	Ensures source authenticity and integrity	Sensor spoofing
mTLS between gateway and ingestion	Mutual authentication	Rogue device/API abuse
AES-256-GCM at rest	Protects stored telemetry	Data exposure
KMS/HSM key management	Key lifecycle and revocation	Key compromise
Immutable audit logs	Tamper-resistant investigation	Incident response
Data lineage tagging	Trace data origin and transformations	Data poisoning

Layer 2 — Network and Perimeter Controls

Control	Purpose	Threat Addressed
Network segmentation	Isolate ingestion, analytics, dashboard	Lateral movement
API gateway + WAF	Inspect and limit API traffic	API exploit

Control	Purpose	Threat Addressed
Private endpoints	Reduce public exposure	External attack surface
Egress filtering	Prevent uncontrolled data exfiltration	Data theft
ZTNA	No implicit network trust	Unauthorized access

Layer 3 — Host, Application, and Endpoint Controls

Control	Purpose	Threat Addressed
Secure container base images	Reduce vulnerable dependencies	Runtime compromise
SAST/DAST in CI/CD	Detect code weaknesses early	Injection flaws
Runtime schema validation	Reject malformed telemetry/API input	Data poisoning/API exploit
ABAC object-level authorization	Prevent horizontal access	Unauthorized data access
EDR on servers/endpoints	Detect malicious behavior	Persistence/lateral movement

Layer 4 — Operational, Managerial, and Monitoring Controls

Control	Purpose	Threat Addressed
SIEM correlation rules	Detect suspicious activity	Multi-stage attacks
SOAR playbooks	Automate response	Incident containment
Incident runbooks	Standardize response	Operational resilience
Security reviews	Governance	Design drift
Periodic access review	Reduce privilege creep	Insider/excess access
Red-team/tabletop exercises	Validate controls	Preparedness

21.17 Defense Architecture PlantUML

```
@startuml
skinparam componentStyle rectangle
title Defense-in-Depth Architecture - Aircraft Telemetry Platform

package "Layer 1: Data Security" {
  [Signed Telemetry]
  [KMS/HSM]
  [Encryption at Rest]
}

package "Layer 2: Network Security" {
  [mTLS]
  [WAF/API Gateway]
  [Network Segmentation]
  [Egress Filtering]
}

package "Layer 3: Application/Host Security" {
  [Schema Validation]
  [ABAC Authorization]
  [Container Scanning]
  [EDR]
}

package "Layer 4: Operations" {
  [SIEM]
  [SOAR]
  [Incident Runbooks]
  [Access Reviews]
}

[Aircraft Edge Gateway] --> [Signed Telemetry]
[Signed Telemetry] --> [mTLS]
[mTLS] --> [WAF/API Gateway]
[WAF/API Gateway] --> [Schema Validation]
[Schema Validation] --> [SIEM]
[SIEM] --> [SOAR]
@enduml
```

21.18 Gap Analysis

Area	Current Simulated State	Proposed Target State	Gap Severity	Remediation
Telemetry authenticity	Gateway sends encrypted telemetry but no payload signature	Per-message/batch signing with device identity	High	Implement signing and verification
API authorization	Role-based access only	ABAC with object-level checks	High	Add policy engine
Monitoring	Logs collected but weak correlation	SIEM correlation and SOAR playbooks	Medium-High	Create detection rules
Network exposure	API partially public	Private endpoints + WAF + ZTNA	Medium	Reduce public exposure
Key management	Manual certificate lifecycle	Automated certificate rotation	Medium	Integrate KMS and cert automation
Data lineage	Partial metadata	Full lineage tags and immutable logs	Medium	Add lineage tracking

21.19 CVSS Quantification

Vulnerability 1 — API Authorization Bypass

Metric	Value	Reason
AV	N	Exploitable over network
AC	L	Low complexity if authorization bug is present
PR	L	Requires low-level authenticated access
UI	N	No user interaction needed
S	U	Impact remains within same security scope
C	H	Telemetry and maintenance data exposed
I	H	API may alter maintenance workflow
A	L	Limited availability impact

Vector:

```
AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L
```

Estimated severity: **High**

Vulnerability 2 — Telemetry Spoofing / Data Poisoning

Metric	Value	Reason
AV	A	Requires adjacent/specialized access to telemetry path or gateway
AC	L	Once access exists, spoofing may be straightforward
PR	N	No platform account required
UI	N	No user interaction needed
S	U	Impact remains in telemetry platform scope
C	N	Main impact is not confidentiality
I	H	Integrity impact is severe
A	H	Incorrect maintenance decisions may affect operations

Vector:

```
AV:A/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:H
```

Estimated severity: **High**

21.20 Enterprise Constraints

Constraint	Explanation	Impact
Legacy avionics integration	Older subsystems may not support modern cryptographic protocols	Limits immediate rollout
Certification and safety review	Changes affecting flight/maintenance systems require rigorous approval	Slower implementation
Downtime windows	Aircraft and fleet operations cannot tolerate long outages	Requires phased deployment
Cost	KMS, SIEM, data lineage, and gateway upgrades require investment	Budget approvals needed
Developer experience	Additional validation and signing increases CI/CD complexity	Training and tooling needed

Constraint	Explanation	Impact
Partner integration	External partners may have different security maturity	Requires contract and API governance

21.21 High-Impact Immediate Remediation

Recommended Next-Sprint Change

Implement **signed telemetry verification in the ingestion service** for all edge gateway telemetry batches.

Justification

Criterion	Assessment
Risk reduction	Very high; directly reduces data poisoning risk
Implementation effort	Medium; can be implemented first at ingestion layer
Business disruption	Low to medium if rolled out in monitor-only mode first
Security value	High; establishes trustworthy data foundation
Long-term benefit	Enables lineage, non-repudiation, and stronger analytics confidence

Phased Rollout

1. Week 1: Define signing format and gateway identity model
2. Week 2: Implement signature verification in ingestion service in monitor-only mode
3. Week 3: Add alerting for unsigned/invalid telemetry
4. Week 4: Enforce rejection for invalid telemetry from selected pilot fleet
5. Later: Expand to all telemetry sources and add automated certificate rotation

---

22. PlantUML Script Library

22.1 C4 Context Template

```
@startuml
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Context.puml

LAYOUT_WITH_LEGEND()

title C4 Level 1 - System Context

Person(user, "Primary User", "Describe role")
System(system, "Target System", "Describe system purpose")
System_Ext(external, "External System", "Describe dependency")

Rel(user, system, "Uses", "HTTPS")
Rel(system, external, "Integrates with", "API")

@enduml
```

22.2 C4 Container Template

```
@startuml
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Container.puml

LAYOUT_WITH_LEGEND()

title C4 Level 2 - Container Diagram

Person(user, "User")

System_Boundary(systemBoundary, "Target System") {
    Container(web, "Web Application", "React/Angular/etc.", "User interface")
    Container(api, "API Gateway", "REST/GraphQL", "API entry point")
    Container(service, "Business Service", "Microservice", "Core logic")
    Container(db, "Database", "SQL/NoSQL", "Stores data")
}

Rel(user, web, "Uses", "HTTPS")
Rel(web, api, "Calls", "HTTPS")
Rel(api, service, "Routes", "mTLS")
Rel(service, db, "Reads/Writes", "Encrypted connection")

@enduml
```

22.3 C4 Component Template

```
@startuml
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Component.puml

LAYOUT_WITH_LEGEND()

title C4 Level 3 - Component Diagram

Container_Boundary(service, "Business Service") {
    Component(auth, "Authorization Module", "Policy engine", "Checks permissions")
    Component(validation, "Input Validator", "Validation library", "Validates requests")
    Component(processor, "Business Processor", "Application logic", "Processes workflow")
    Component(audit, "Audit Logger", "Logging library", "Writes audit events")
}

Rel(auth, processor, "Authorizes")
Rel(validation, processor, "Validates input for")
Rel(processor, audit, "Logs actions")

@enduml
```

22.4 Data Flow Diagram Template

```
@startuml
title Data Flow and Trust Boundaries

actor User
rectangle "Public Zone" {
  component "Web App" as Web
}
rectangle "Private Application Zone" {
  component "API Gateway" as API
  component "Service" as Service
}
database "Data Store" as DB
cloud "External Partner" as Partner

User --> Web : HTTPS
Web --> API : HTTPS
API --> Service : mTLS
Service --> DB : Encrypted DB connection
Service --> Partner : mTLS API

note right of API
Trust boundary:
Public to private entry point
end note

@enduml
```

22.5 Defense-in-Depth Diagram Template

```
@startuml
title Defense-in-Depth Control Stack

package "Data Security" {
  [Encryption]
  [Signing]
  [KMS]
}
package "Network Security" {
  [WAF]
  [mTLS]
  [Segmentation]
}
package "Application Security" {
  [Input Validation]
  [Authorization]
  [SAST/DAST]
}
package "Operations" {
  [SIEM]
  [SOAR]
  [Incident Response]
}

[Threat] --> [WAF]
[WAF] --> [Input Validation]
[Input Validation] --> [Authorization]
[Authorization] --> [SIEM]
@enduml
```

22.6 Risk Heatmap Template

```
@startuml
title Risk Heatmap Concept

rectangle "High Impact / High Likelihood" #FF9999
rectangle "High Impact / Low Likelihood" #FFD699
rectangle "Low Impact / High Likelihood" #FFFF99
rectangle "Low Impact / Low Likelihood" #CCFFCC

@enduml
```

---

23. Student GitHub Starter Repository Template

23.1 README.md

````
# Advanced Cyber Security Assignment

## Student Information
- Name:
- Student ID:
- Course: SS*ZG681 / SE*ZG681 Advanced Cyber Security
- System selected:
- System type: Real / Anonymized / Simulated

## Confidentiality Statement
This repository does not contain proprietary, sensitive, or confidential organizational information. All sensitive details have been ano

## Repository Structure

```text
report/        Final report source and PDF
diagrams/      PlantUML source files
risk/          CVSS tables and risk register
scripts/       Structure and grading helper scripts
presentation/  Viva presentation outline
````

How to Render Diagrams

Use PlantUML locally or through an approved tool.

Submission Checklist

- [ ] Final report PDF included
- [ ] Markdown report included
- [ ] C4 context diagram included
- [ ] C4 container diagram included
- [ ] C4 component diagram included
- [ ] Two attack graphs included
- [ ] CVSS analysis included
- [ ] Reflection included
````

````
## 23.2 .gitignore

```gitignore
# OS files
.DS_Store
Thumbs.db

# Python
__pycache__/
*.pyc
.venv/
venv/

# Generated outputs
*.tmp
*.log

# Sensitive files - do not commit
.env
*.key
*.pem
*.p12
credentials.*
secrets.*
````

23.3 risk/risk-register.csv Template

```
RiskID,Threat,Asset,Impact,Likelihood,CVSS_Vector,CVSS_Score,Existing_Control,Proposed_Control,Residual_Risk
R1,API authorization bypass,Telemetry API,High,Medium,AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L,High,RBAC,ABAC object-level authorization,Medi
R2,Telemetry spoofing,Telemetry pipeline,Critical,Medium,AV:A/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:H,High,TLS,Signed telemetry,Low
```

23.4 presentation/viva-presentation-outline.md Template

```
# Viva Presentation Outline

## Slide 1: Title
## Slide 2: System Overview
## Slide 3: C4 Context and Container Diagrams
## Slide 4: Data Classification and CIA Impact
## Slide 5: Threat Scenario 1
## Slide 6: Threat Scenario 2
## Slide 7: Defense-in-Depth Design
## Slide 8: CVSS and Gap Analysis
## Slide 9: Remediation Roadmap
## Slide 10: Reflection and Lessons Learned
```

---

24. Automated Grading Helper Scripts

These scripts are provided as optional helper tools. They do not replace instructor judgment. They check structure, completeness, and presence of required elements.

24.1 scripts/check_structure.py
```

```
from pathlib import Path

REQUIRED_PATHS = [
    "README.md",
    "report/final-report.md",
    "diagrams/c4-context.puml",
    "diagrams/c4-container.puml",
    "diagrams/c4-component.puml",
    "diagrams/attack-graph-scenario-1.puml",
    "diagrams/attack-graph-scenario-2.puml",
    "risk/cvss-table.md",
    "risk/risk-register.csv",
    "presentation/viva-presentation-outline.md",
]

OPTIONAL_PATHS = [
    "diagrams/c4-code-optional.puml",
    "diagrams/did-architecture.puml",
    "report/final-report.pdf",
]


def main():
    root = Path.cwd()
    missing = []
    present = []

    for item in REQUIRED_PATHS:
        path = root / item
        if path.exists():
            present.append(item)
        else:
            missing.append(item)

    print("\n=== Assignment Structure Check ===")
    print(f"Present required files: {len(present)} / {len(REQUIRED_PATHS)}")

    if present:
        print("\nPresent:")
        for item in present:
            print(f"  [OK] {item}")

    if missing:
        print("\nMissing:")
        for item in missing:
            print(f"  [MISSING] {item}")
    else:
        print("\nAll required files are present.")

    print("\nOptional files:")
    for item in OPTIONAL_PATHS:
        status = "OK" if (root / item).exists() else "not found"
        print(f"  [{status}] {item}")

    if missing:
        raise SystemExit(1)


if __name__ == "__main__":
    main()
```

24.2 scripts/grade_report.py

```
from pathlib import Path
import re

REPORT_PATH = Path("report/final-report.md")

REQUIRED_KEYWORDS = {
    "C4 Context": ["context diagram", "c4"],
    "C4 Container": ["container diagram", "c4"],
    "C4 Component": ["component diagram", "c4"],
    "Data Classification": ["data classification"],
    "CIA Analysis": ["confidentiality", "integrity", "availability"],
    "Threat Modeling": ["attack graph", "initial access", "lateral movement"],
    "Defense in Depth": ["defense-in-depth", "cryptographic", "network", "application", "operational"],
    "CVSS": ["cvss", "av:", "ac:", "pr:", "ui:"],
    "Reflection": ["constraints", "trade-off", "remediation"],
}

SECTION_SCORES = {
    "Architecture": 5,
    "CIA and Crypto": 3,
    "Threat Modeling": 5,
    "Defense Design": 7,
    "Gap and CVSS": 5,
    "Reflection": 5,
}


def keyword_present(text, keywords):
    text_lower = text.lower()
    return all(keyword.lower() in text_lower for keyword in keywords)


def estimate_completeness(text):
    results = {}
    for label, keywords in REQUIRED_KEYWORDS.items():
        results[label] = keyword_present(text, keywords)
    return results


def main():
    if not REPORT_PATH.exists():
        print(f"Report not found: {REPORT_PATH}")
        raise SystemExit(1)

    text = REPORT_PATH.read_text(encoding="utf-8")
    word_count = len(re.findall(r"\w+", text))
    completeness = estimate_completeness(text)

    print("\n=== Automated Completeness Review ===")
    print(f"Word count: {word_count}")

    print("\nRequired topic presence:")
    for label, found in completeness.items():
        print(f"  {'[OK]' if found else '[CHECK]'} {label}")

    print("\nSuggested instructor scoring template:")
    for section, marks in SECTION_SCORES.items():
        print(f"  {section}: ___ / {marks}")

    print("\nNote: This script checks structure only. It does not judge technical correctness or originality.")


if __name__ == "__main__":
    main()
```

24.3 scripts/check_plantuml_files.py

```
from pathlib import Path

DIAGRAM_DIR = Path("diagrams")


def main():
    if not DIAGRAM_DIR.exists():
        print("Missing diagrams directory.")
        raise SystemExit(1)

    puml_files = list(DIAGRAM_DIR.glob("*.puml"))
    print(f"Found {len(puml_files)} PlantUML files.")

    for file in puml_files:
        text = file.read_text(encoding="utf-8", errors="ignore")
        has_start = "@startuml" in text
        has_end = "@enduml" in text
        status = "OK" if has_start and has_end else "CHECK"
        print(f"[{status}] {file}")

    if len(puml_files) < 5:
        print("Expected at least 5 PlantUML diagrams: context, container, component, and two attack graphs.")
        raise SystemExit(1)


if __name__ == "__main__":
    main()
```

24.4 GitHub Actions Workflow Optional

```
name: Assignment Structure Check

on:
  push:
  pull_request:

jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Run structure check
        run: python scripts/check_structure.py

      - name: Run report completeness check
        run: python scripts/grade_report.py

      - name: Run PlantUML file check
        run: python scripts/check_plantuml_files.py
```

25. Originality-Safe Project Variants and Academic Integrity Guidance

25.1 Important Academic Integrity Statement

Students must submit original work. The examples in this document are learning references, not answers to copy. Students may adapt the structure but must use their own system, assumptions, analysis, diagrams, attack scenarios, controls, and reflections.

The purpose of the variants below is to encourage diversity of thinking and reduce repeated, generic submissions. They are not intended to bypass plagiarism detection. Students should cite references, acknowledge assumptions, and clearly identify simulated content.

25.2 How to Create an Original Variant

Students can vary:

- System domain
- Data assets
- Attack scenarios
- Trust boundaries
- Cloud/on-prem/edge architecture
- Compliance assumptions
- Threat actor type
- Control priorities
- Remediation strategy

25.3 Variant Matrix

Variant	Domain	Primary Risk	Suggested Threats	Suggested Controls
A	Predictive maintenance	Telemetry integrity	Sensor spoofing, data poisoning	Signed telemetry, data lineage
B	Passenger platform	PII exposure	API exploit, session hijacking	OIDC hardening, WAF, DLP
C	Flight operations	Availability	DDoS, message replay	Redundancy, replay prevention
D	Partner maintenance API	Third-party risk	Partner compromise, token misuse	mTLS, scoped tokens, contract controls
E	Data lake analytics	Insider/data abuse	Excessive access, model poisoning	ABAC, lineage, monitoring
F	Edge gateway firmware	Device compromise	Firmware tampering, rogue update	Secure boot, signed firmware

25.4 Originality Checklist

- [ ] I selected my own system or clearly declared a simulated system.
- [ ] I wrote my own architecture narrative.
- [ ] My diagrams are customized to my system.
- [ ] My attack scenarios are specific to my architecture.
- [ ] My CVSS scoring includes my own reasoning.
- [ ] My reflection includes realistic constraints from my chosen scenario.
- [ ] I did not copy the gold solution as my final answer.

26. Optional Enrichment Activities

These are optional but can make the assignment a memorable learning experience.

26.1 Security Architecture Review Board Simulation

Students present their architecture to a mock review board consisting of:

- Security architect
- Product owner
- Operations lead
- Compliance officer
- Developer lead

Each reviewer asks questions from their viewpoint.

26.2 Red Team vs Blue Team Mini Exercise

Students pair up:

- Student A attacks Student B's architecture using attack graphs.
- Student B improves defenses using control mapping.

26.3 One-Page Executive Risk Memo

Students write a one-page memo for leadership:

- Top 2 risks
- Business impact
- Recommended investment
- Expected risk reduction

26.4 Architecture Decision Record

Students include one ADR for a major security decision.

Template:

```
# ADR-001: Adopt Signed Telemetry at Edge Gateway

## Status
Proposed / Accepted

## Context
Telemetry integrity is critical.

## Decision
Use device-bound keys to sign telemetry batches.

## Consequences
Improves integrity but adds certificate lifecycle complexity.
```

26.5 Security Control Traceability Challenge

Students must prove that every proposed control maps to at least one threat stage.

---

27. Final Submission Checklist

27.1 Report Checklist

- ☐ Cover page completed
- ☐ Confidentiality statement included
- ☐ Executive summary included
- ☐ C4 context diagram included
- ☐ C4 container diagram included
- ☐ C4 component diagram included
- ☐ Data flow table included
- ☐ Data classification matrix included
- ☐ CIA analysis included
- ☐ Cryptographic policy included
- ☐ Two attack scenarios included
- ☐ Attack graphs included
- ☐ Defense-in-depth design included
- ☐ Gap analysis included
- ☐ CVSS vectors included
- ☐ Enterprise constraints included
- ☐ High-impact remediation included
- ☐ References/assumptions included

27.2 GitHub Checklist

- ☐ README present
- ☐ Report Markdown present
- ☐ Report PDF present
- ☐ PlantUML files present
- ☐ Risk register present
- ☐ Presentation outline present
- ☐ No secrets committed
- ☐ Commit history meaningful

27.3 Diagram Checklist

- [ ] Diagrams use PlantUML or another code-based tool
- [ ] C4 model followed
- [ ] Trust boundaries shown
- [ ] Protocols shown where relevant
- [ ] Security controls shown where relevant
- [ ] Diagram source files included

---

28. Appendix A: Full Markdown Report Template

```
# Advanced Cyber Security Assignment Report

## Cover Page
[Student details]

## Confidentiality Statement
[Safe abstraction declaration]

## Executive Summary
[Summary]

## B.2 Anchor — System and Threat Surface Blueprinting
### System Description
### Runtime Architecture
### C4 Diagrams
### Data Flow Mapping
### Data Classification Matrix
### Compliance Boundary

## B.3 Apply — Advanced Cyber Security Engineering Pipeline
### Task 1: CIA and Cryptographic Policy
### Task 2: Threat Modeling and Attack Graphs
### Task 3: Defense-in-Depth Architecture

## B.4 Analyse — Gap Analysis and CVSS Quantification
### Current State
### Proposed State
### Gap Analysis
### CVSS Scoring
### Leadership Interpretation

## B.5 Reflect — Constraints and Lifecycle Maintenance
### Technical Constraints
### Organizational Constraints
### TCO
### Developer Experience
### Immediate Remediation

## References and Assumptions
```

---

29. Appendix B: README Template

```
# SS*ZG681 / SE*ZG681 Advanced Cyber Security Assignment

## Project Title
[Title]

## Student
[Name and ID]

## System Type
Real / Anonymized / Simulated

## Confidentiality Statement
No proprietary details are included.

## Contents
- report/final-report.md
- report/final-report.pdf
- diagrams/*.puml
- risk/*.md or *.csv
- presentation/viva-presentation-outline.md

## How to Review
1. Read report/final-report.pdf
2. Review diagrams/*.puml
3. Review risk/cvss-table.md
4. Review presentation/viva-presentation-outline.md
```

---

30. Appendix C: Sample Commit Plan

Week	Commit Goal	Example Commit Message
Week 1	Initial repo and system selection	`init: add assignment structure and selected system`
Week 2	C4 diagrams	`docs: add C4 context and container diagrams`
Week 3	Data classification and CIA	`docs: add data classification and CIA analysis`
Week 4	Threat modeling	`docs: add attack graphs and threat scenarios`
Week 5	Defense design	`docs: add defense-in-depth controls`
Week 6	Gap and CVSS	`docs: add CVSS and gap analysis`
Week 7	Reflection and final polish	`docs: finalize report and viva outline`

---

31. Appendix D: Suggested Tools

Purpose	Preferred Tool	Alternatives
Diagrams-as-code	PlantUML	Mermaid, Structurizr DSL
Version control	GitHub	GitLab, Bitbucket
Markdown editing	VS Code	Obsidian, Typora
PDF conversion	Pandoc	VS Code extensions, Markdown export tools
CVSS calculation	Official CVSS calculator / spreadsheet	Manual table

Purpose	Preferred Tool	Alternatives
Threat modeling	Markdown + PlantUML	OWASP Threat Dragon
Security references	OWASP, MITRE, NIST, vendor docs	Internal approved documentation

---

32. Appendix E: Common Mistakes to Avoid

Mistake	Why It Is a Problem	Better Approach
Drawing only one high-level diagram	Insufficient architecture depth	Include C4 context, container, component
Listing generic threats	Does not show system understanding	Build threats from your actual architecture
Ignoring integrity	Many cyber-physical systems depend on data integrity	Analyze spoofing, tampering, poisoning
No CVSS reasoning	Score appears arbitrary	Explain each vector metric
Copying sample solution	Violates originality expectations	Use samples as structure only
Over-disclosing company details	Confidentiality risk	Abstract and anonymize
Controls not mapped to threats	Weak engineering logic	Use traceability matrix
Reflection too generic	Misses WILP professional context	Discuss real constraints and trade-offs

---

Final Message to Students

This assignment is designed to help you think like a security architect, not merely like a checklist auditor. A strong submission will show that you understand how systems fail, how attackers move, how controls interact, and how real organizations balance security with operational realities.

Use the templates, but make the work your own. Use PlantUML and GitHub to make your thinking reproducible. Use C4 diagrams to communicate architecture clearly. Use CVSS to make risk visible. Use reflection to show professional maturity.

A memorable submission is one where the reader can say:

> “This student understands the system, the threat, the engineering trade-offs, and the business reality.”

End of Unified Assignment Handbook